

NAME: GOLDA MEIR KABUKI AMETEPEY

INDEX NO: 1725147399

COURSE: JAVA PROGRAMMING (ASSIGNMENT)

BIT L300

QUESTION

Q2

A. Create a class name **Emolument**, to represent an emolument. The class contains the following:

i. Two double data fields name **basic_salary** and **tax_relief** that represent basic salary and tax relief amounts. This data fields should be encapsulated.

2 Marks

ii. A constructor that creates emolument with the specified Basic Salary and Tax Relief.

3 Marks

iii. A method named **getBasicsalary()** that returns the basic_salary.

1 Mark

iv. A method named **getTaxRelief()** that returns the tax_relief.

1 Mark

v. A method named **SSNIT()** that returns the computed SSNIT contribution of the emolument. Note: SSNIT contribution is 3.5% of Basic Salary.

2 Marks

vi. A method named **taxableIncome()** that returns the computed Taxable Income of the emolument. Note: Taxable Income = Basic salary – (Tax relief + SSNIT contribution).

2 Marks

B. Create a class named **MyEmolument** that inherits from Emolument class. The class contains the following:

i. Two double data fields name **basic_salary** and **tax_relief**, that represent basic salary and tax relief amounts. This data fields should be encapsulated.

2 Marks

ii. A **non-arg constructor** that creates a default emolument. The default values are **0** for basic salary and tax relief.

2 Marks

iii. A constructor that creates **MyEmolument** with the specified basic salary and Tax relief.

1 Mark

iv. A method named **incomeTax()** that return the computed Income Tax. Income Tax is calculated as follows:

a. The first 500.00 of Taxable Income, the tax rate is 5%.

- b. The next 500.00 Taxable Income, the tax rate is 12.5%.
- c. The rest, the tax rate is 17.5%.

3 Marks

- v. A method named **totalDeduction()** that return the value of Total Deduction.
Note: *Total Deduction = SSNIT Contribution + Income Tax.*

1 Mark

- vi. A method named **netSalary()** that return the value of Net Salary.
Note: *Net Salary = Basic salary – Total Deduction.*

1 Mark

C. Write a test program that creates MyEmolument object named **Staff_Salary** with below requirement.

- i. Accept input from the user using the input dialog.

2 Marks

- ii. Display the Basic Salary, Tax Relief, SSNIT Contribution, Taxable Income, Income Tax, Total Deduction and Net Salary as shown in the figure below.

7 Marks

ANSWERS

```
class Emolument:
```

```
    def __init__(self, basic_salary, tax_relief):
```

```
        self.__basic_salary = basic_salary
```

```
        self.__tax_relief = tax_relief
```

```
    def getBasicsalary(self):
```

```
        return self.__basic_salary
```

```
    def getTaxRelief(self):
```

```
        return self.__tax_relief
```

```
def SSNIT(self):
```

```
    return self.__basic_salary * 0.035
```

```
def taxableIncome(self):
```

```
    return self.__basic_salary - (self.__tax_relief + self.SSNIT())
```

```
class MyEmolument(Emolument):
```

```
    def __init__(self, basic_salary=0, tax_relief=0):
```

```
        super().__init__(basic_salary, tax_relief)
```

```
    def incomeTax(self):
```

```
        taxable_income = self.taxableIncome()
```

```
        tax = 0
```

```
        if taxable_income <= 500:
```

```
            tax = taxable_income * 0.05
```

```
        elif taxable_income <= 1000:
```

```
            tax = 500 * 0.05 + (taxable_income - 500) * 0.125
```

```
        else:
```

```
            tax = 500 * 0.05 + 500 * 0.125 + (taxable_income - 1000) * 0.175
```

```
        return tax
```

```
    def totalDeduction(self):
```

```
        return self.SSNIT() + self.incomeTax()
```

```
def netSalary(self):

    return self.getBasicsalary() - self.totalDeduction()


# Test Program

def main():

    print("Enter details for Staff Salary:")

    basic_salary = float(input("Enter Basic Salary: "))

    tax_relief = float(input("Enter Tax Relief: "))

    # Create MyEmolument object

    Staff_Salary = MyEmolument(basic_salary, tax_relief)

    # Display results

    print("\n--- Salary Breakdown ---")

    print(f"Basic Salary: {Staff_Salary.getBasicsalary():.2f}")

    print(f"Tax Relief: {Staff_Salary.getTaxRelief():.2f}")

    print(f"SSNIT Contribution: {Staff_Salary.SSNIT():.2f}")

    print(f"Taxable Income: {Staff_Salary.taxableIncome():.2f}")

    print(f"Income Tax: {Staff_Salary.incomeTax():.2f}")

    print(f"Total Deduction: {Staff_Salary.totalDeduction():.2f}")

    print(f"Net Salary: {Staff_Salary.netSalary():.2f}")

if __name__ == "__main__":

    main()
```

Explanation

Class Emolument

1. **Encapsulation:** basic_salary and tax_relief are private attributes.
2. **Constructor:** Accepts basic_salary and tax_relief.
3. **Methods:**
 - getBasicsalary(): Returns the basic salary.
 - getTaxRelief(): Returns the tax relief.
 - SSNIT(): Calculates 3.5% of basic_salary.
 - taxableIncome(): Computes taxable income as basic_salary - (tax_relief + SSNIT).

Class MyEmolument

1. **Inherits from Emolument:**
 - Default constructor initializes basic_salary and tax_relief to 0.
 - Parameterized constructor initializes them with given values.
2. **Additional Methods:**
 - incomeTax(): Computes income tax based on brackets.
 - totalDeduction(): Calculates total deductions as SSNIT + Income Tax.
 - netSalary(): Computes net salary as basic_salary - totalDeduction.

Test Program

- Prompts the user for basic_salary and tax_relief.
- Displays the full breakdown: Basic Salary, Tax Relief, SSNIT Contribution, Taxable Income, Income Tax, Total Deduction, and Net Salary.